

IIT Tutorial Week 6 - Sem 2, 2026

This tutorial is split into two parts. Part A covers core Python function skills. Part B introduces the Tkinter GUI toolkit and connects buttons and inputs to your functions.

Learning goals

- ✓ Write and call functions with parameters and return values.
- ✓ Use docstrings, default/keyword arguments, and basic scope.
- ✓ Build a simple GUI with labels, buttons, and text entries.
- ✓ Connect GUI events to Python functions (callbacks).

Setup

- Install **Python 3.x** (Windows/Mac/Linux).
- Use **IDLE** or your favorite editor.
- Tkinter is included with most Python installs. On Windows with official Python, it should work out of the box.

Part A — Python Functions

1) Why functions?

Functions group steps into reusable blocks. They make your code shorter, clearer, and easier to test.

```
# A simple function
def greet(name):
```

```
# Return a friendly greeting:
return f"Hello, {name}!"

# Using the function
message = greet("Aisha")
print(message) # Hello, Aisha!
```

2) Parameters and return values

```
def area_of_rectangle(width, height):
    return width * height

print(area_of_rectangle(3, 5)) # 15
```

3) Default & keyword arguments

```
def power(base, exponent=2):
    return base ** exponent

print(power(5)) # 25 (uses default exponent=2)
print(power(2, exponent=3)) # 8 (keyword argument)
```

4) Docstrings & scope

```
PI = 3.14159 # global variable

def circle_area(r):
    "Compute area of a circle of radius r."
    return PI * (r ** 2)

print(circle_area(2)) # 12.56...
```

□ Quick Exercises

```
def fahrenheit_to_celsius(f):  
    return (f - 32) * 5/9  
temp = fahrenheit_to_celsius(68)  
print(f"Celsius is: {temp:.2f}")
```

```
def is_even(n):  
    return n % 2 == 0  
number = is_even(10)  
print(f"The number is even: {number}")
```

```
def triangle_area(b, h=1):  
    return 0.5 * b * h  
area = triangle_area(4, 5)  
print(f"The area is: {area:.2f}")
```

1. Write `fahrenheit_to_celsius(f)` that returns °C.
2. Write `is_even(n)` that returns True if n is even.
3. Write `triangle_area(b, h=1)` with default height 1.

► **Show sample solutions**

Part B — Tkinter GUI Basics

Tkinter lets you build desktop apps using Python. You create a window, add widgets (labels, buttons, text boxes), and connect them to your functions.

1) Smallest Tkinter app

```
import tkinter as tk  
  
root = tk.Tk() # create the main window  
root.title("My First App")  
root.geometry("320x160") # width x height (optional)  
  
label = tk.Label(root, text="Hello, Tkinter!")  
label.pack(pady=10)  
  
root.mainloop() # start the event loop
```

2) Buttons and callbacks

```
import tkinter as tk  
  
def say_hello():  
    print("Hello from the button!")  
  
root = tk.Tk()  
root.title("Callbacks")
```

```
btn = tk.Button(root, text="Click me", command=say_hello)
btn.pack(pady=10)

root.mainloop()
```

3) Entry + Label (read input)

```
import tkinter as tk

def greet():
    name = name_entry.get()
    output.config(text=f"Hello, {name or 'friend'}!")

root = tk.Tk()
root.title("Greeter")

tk.Label(root, text="Your name:").pack()
name_entry = tk.Entry(root, width=24)
name_entry.pack()

tk.Button(root, text="Greet", command=greet).pack(pady=6)

output = tk.Label(root, text="")
output.pack()

root.mainloop()
```

4) Layout with grid

```
import tkinter as tk

root = tk.Tk()
root.title("Grid Example")

tk.Label(root, text="Row 0, Col 0").grid(row=0, column=0,
```

```

tk.Label(root, text="Row 0, Col 1").grid(row=0, column=1,

tk.Button(root, text="A").grid(row=1, column=0, sticky="e
tk.Button(root, text="B").grid(row=1, column=1, sticky="e

root.columnconfigure(0, weight=1)
root.columnconfigure(1, weight=1)

root.mainloop()

```

5) Mini-project: Temperature Converter (°F → °C)

We'll reuse the function from Part A and connect it to a GUI.

```

import tkinter as tk

def fahrenheit_to_celsius(f):
    return (f - 32) * 5/9

def convert():
    try:
        f = float(entry_f.get())
        c = fahrenheit_to_celsius(f)
        result_var.set(f"{c:.2f} °C")
        status_var.set("Converted successfully.")
    except ValueError:
        result_var.set("")
        status_var.set("Please enter a valid number.")

root = tk.Tk()
root.title("Temperature Converter")
root.geometry("360x160")

tk.Label(root, text="Fahrenheit:").grid(row=0, column=0,
entry_f = tk.Entry(root, width=12)
entry_f.grid(row=0, column=1, padx=8, pady=8, sticky="w")

```

```
tk.Button(root, text="Convert", command=convert).grid(row=0, column=1)

tk.Label(root, text="Celsius:").grid(row=1, column=0, padx=10)
result_var = tk.StringVar(value="")
tk.Label(root, textvariable=result_var).grid(row=1, column=1)

status_var = tk.StringVar(value="")
tk.Label(root, textvariable=status_var, fg="green").grid(row=2, column=1)

root.columnconfigure(1, weight=1)
root.mainloop()
```

```
import tkinter as tk
```

```
count = 0
def increment():
    global count
    count += 1
    label.config(text=str(count))
```

```
root = tk.Tk()
root.title("Click Counter")
label = tk.Label(root, text="0")
label.pack()
```

```
button = tk.Button(root, text="Add 1", command=increment)
button.pack()
root.mainloop()
```

▣ Build It Yourself: Click Counter

Create a window with a label that shows a number starting at 0 and a button that increases the number by 1 each click.

► Show a sample implementation

6) Common patterns & good habits

Separate logic & UI

Put calculations (functions) in one place and GUI wiring in another. Easier to test and reuse.

Use StringVar / IntVar

These special variables link widget values to Python variables in real time.

```
name_var = tk.StringVar()
name_entry = tk.Entry(ro
```

Don't block the event loop

A long time.sleep() will freeze your GUI. Offload heavy work to threads or use after().

```
import tkinter as tk
from tkinter import ttk
```

```
def calculate(a, b, op):
    if op == "+":
        return a + b
    elif op == "-":
        return a - b
    elif op == "x":
        return a * b
    elif op == "÷":
        return a / b
```

```
def compute():
    try:
        a = float(entry_a.get())
        b = float(entry_b.get())
        op = operation_var.get()
```

```
        if op == "÷" and b == 0:
            result_var.set("Cannot divide by zero.")
        else:
            result = calculate(a, b, op)
            result_var.set(f"Result: {result: 2f}")
```

```
    except ValueError:
        result_var.set("Please enter valid numbers.")
```

7) Challenge: Mini-Calculator

Make a small GUI with two number entries, an operation dropdown (+, -, ×, ÷), and a “Compute” button that shows the result using a function you wrote.



Hint

Extras

- Try replacing pack() with grid() or place() to see different layout strategies.
- Add icons and change the window size with root.iconbitmap() and root.geometry().
- Keyboard shortcut: bind Enter to the Convert button with root.bind("<Return>", lambda e: convert()).

```
root = tk.Tk()
root.title("Mini Calculator")
root.geometry("360x220")

tk.Label(root, text="First number:").grid(row=0, column=0, padx=8, pady=8)
entry_a = tk.Entry(root, width=12)
entry_a.grid(row=0, column=1, padx=8, pady=8)

tk.Label(root, text="Second number:").grid(row=1, column=0, padx=8, pady=8)
entry_b = tk.Entry(root, width=12)
entry_b.grid(row=1, column=1, padx=8, pady=8)

tk.Label(root, text="Operation:").grid(row=2, column=0, padx=8, pady=8)
operation_var = tk.StringVar()
operation = ttk.Combobox(
    root,
    textvariable=operation_var,
    values=["+", "-", "x", "÷"],
    state="readonly",
    width=10
)
operation.grid(row=2, column=1, padx=8, pady=8)

tk.Button(root, text="Compute", command=compute).grid(
    row=3, column=0, columnspan=2, pady=8
)

result_var = tk.StringVar(value="")
tk.Label(root, textvariable=result_var).grid(
    row=4, column=0, columnspan=2, pady=8
)

root.mainloop()
```

Case Study — Student Marks Calculator (Functions + GUI)

Let’s combine both skills: write a function to calculate a student’s average grade and connect it to a Tkinter GUI.

Problem

You are asked to build a small desktop app for teachers. The teacher can enter three marks (out of 100). The app should calculate the average and show whether the student has **Passed** (≥ 50) or **Failed**.

Step 1 — The Function

```
def calculate_average(mark1, mark2, mark3):
    average = (mark1 + mark2 + mark3) / 3

    if average >= 50:
        result = "Passed"
    else:
        result = "Failed"

    return average, result
```

Step 2 — The GUI

```
import tkinter as tk
```

```
root.mainloop()
```

💡 Extension Ideas

- Allow entering any number of marks (hint: use a list).
- Add a “Clear” button to reset the fields.
- Change the background colour: green if Passed, red if Failed.

© University of Canberra • Faculty of Science and Technology

```
import tkinter as tk
```

```
def calculate_average(mark1, mark2, mark3):
```

```
    average = (mark1 + mark2 + mark3) / 3
```

```
    if average >= 50:
```

```
        status = "Passed"
```

```
    else:
```

```
        status = "Failed"
```

```
    return average, status
```

```
def calculate():
```

```
    try:
```

```
        mark1 = float(entry1.get())
```

```
        mark2 = float(entry2.get())
```

```
        mark3 = float(entry3.get())
```

```
        average, status = calculate_average(mark1, mark2, mark3)
```

```
        average_var.set(f"Average: {average:.2f}")
```

```
        status_var.set(f"Status: {status}")
```

```
        if status == "Passed":
```

```
            status_label.config(fg="green")
```

```
        else:
```

```
            status_label.config(fg="red")
```

```
    except ValueError:
```

```
        average_var.set("")
```

```
        status_var.set("Please enter valid marks.")
```

```
        status_label.config(fg="black")
```

```
root = tk.Tk()
```

```
root.title("Student Marks Calculator")
```

```
root.geometry("360x250")
```

```
tk.Label(root, text="Mark 1:").grid(row=0, column=0, padx=8, pady=8)
```

```
entry1 = tk.Entry(root, width=12)
```

```
entry1.grid(row=0, column=1, padx=8, pady=8)
```

```
tk.Label(root, text="Mark 2:").grid(row=1, column=0, padx=8, pady=8)
```

```
entry2 = tk.Entry(root, width=12)
```

```
entry2.grid(row=1, column=1, padx=8, pady=8)
```

```
tk.Label(root, text="Mark 3:").grid(row=2, column=0, padx=8, pady=8)
```

```
entry3 = tk.Entry(root, width=12)
```

```
entry3.grid(row=2, column=1, padx=8, pady=8)
```

```
tk.Button(root, text="Calculate", command=calculate).grid(
    row=3, column=0, columnspan=2, pady=10
)
```

```
average_var = tk.StringVar()
```

```
tk.Label(root, textvariable=average_var).grid(
    row=4, column=0, columnspan=2, pady=5
)
```

```
status_var = tk.StringVar()
```

```
status_label = tk.Label(root, textvariable=status_var)
```

```
status_label.grid(row=5, column=0, columnspan=2, pady=5)
```

```
root.mainloop()
```